

Regression-Linear

April 2, 2026

1 Regression Tutorial

We will learn:

- What regression is
- Why we split data into training and testing sets
- How `LinearRegression()` works
- What R^2 and **Mean Squared Error** mean
- A real-life example using delivery times

1.1 Introduction

Regression is a method used to predict a numeric value, such as house price, salary, or delivery time.

$$y = f(x)$$

Where:

- x = input
- y = value we want to predict

Linear Regression Linear Regression is the simplest type of regression. It assumes the relationship between input and output is a straight line.

$$y = b_0 + b_1x$$

Where:

- b_0 = intercept (starting value)
- b_1 = slope (how much (y) changes when (x) changes)

Advantages - Simple and easy to understand - Fast to train - Low risk of overfitting

Disadvantages - Only works well when the relationship is roughly linear

Ridge Regression (L2)

- Adds **L2 penalty** to shrink coefficients.
- Reduces overfitting but **keeps all features**.
- Useful when predictors are correlated.

- Parameter: **alpha** (strength of regularization)
 - Small alpha → model behaves like Linear Regression
 - Large alpha → simpler model, but may underfit

Lasso Regression (L1)

- Adds **L1 penalty** to shrink some coefficients to **exactly zero**.
- Performs **feature selection** automatically.
- Good when only a few features are important.
- Parameter: **alpha** (strength of regularization)
 - Small alpha → model behaves like Linear Regression
 - Large alpha → simpler model, but may underfit

ElasticNet

- Combines **L1 + L2 penalties** (Lasso + Ridge).
- Useful when many features are correlated.
- Parameters:
 - **alpha**: overall regularization strength
 - **l1_ratio**: balance between L1 and L2
 - * 0 → mostly Ridge
 - * 1 → mostly Lasso

1.2 Real-Life Scenario Example

Imagine you own a delivery company.

You want to predict how long a delivery will take based on:

- Distance traveled
- Gas price
- Number of stops

This is a regression problem because the result is a number (delivery time).

```
[1]: import pandas as pd
df=pd.read_csv('../shared/data/DeliveryTime.csv')
df.head()
```

```
[1]:  milesTraveled  numDeliveries  gasPrice  travelTime
0             89              4       3.84         7.0
1             66              1       3.19         5.4
2             78              3       3.78         6.6
3            111              6       3.39         7.4
4             44              1       3.57         4.8
```

Correlation `pearsonr()` checks whether two variables are related.

- Close to 1 → strong positive relationship

- Close to -1 → strong negative relationship
- Close to 0 → almost no relationship

Real-life meaning: If milesTraveled and deliveryTime have a strong positive correlation, then longer trips usually take more time.

```
[6]: # import numpy as np
# np.cov(df.milesTraveled, df.gasPrice)
from scipy import stats
stats.pearsonr(df.travelTime, df.gasPrice)
stats.pearsonr(df.travelTime, df.numDeliveries)
stats.pearsonr(df.milesTraveled, df.travelTime)
```

```
[6]: PearsonRResult(statistic=0.970943502436054, pvalue=1.8253918485694088e-31)
```

Features and Target

- X contains the input variables (features).
- Y contains the value we want to predict.

```
[8]: df.head()
```

```
[8]:
```

	milesTraveled	numDeliveries	gasPrice	travelTime
0	89	4	3.84	7.0
1	66	1	3.19	5.4
2	78	3	3.78	6.6
3	111	6	3.39	7.4
4	44	1	3.57	4.8

```
[10]: # Determine X and Y
X=df.iloc[:, :-1]
Y=df.iloc[:, -1]
```

Data Split We split the data into training and testing sets so we can check whether the model works on new, unseen data.

- Training set: used to teach the model
- Test set: used to evaluate the model afterward

If we train and test on the same data, the model may simply memorize the answers. It can look very accurate even though it would fail on new examples. This is called overfitting.

Typical split:

- 80% for training
- 20% for testing

```
[13]: # Split data into Training and Test
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2)
```

Linear Regression

- Linear Regression tries to draw the best straight line through the data.
- The model learns something like:

$$\text{delivery_time} = b_0 + b_1(\text{miles}) + b_2(\text{gas_price})$$

where:

- b_0 = starting value
- b_1, b_2 = how much each variable affects delivery time

```
[16]: # Create the model
# from sklearn.linear_model import LinearRegression
# model = LinearRegression()

# from sklearn.linear_model import Ridge
# model = Ridge(alpha=1)

# from sklearn.linear_model import Lasso
# model = Lasso(alpha=1)

from sklearn.linear_model import ElasticNet
model = ElasticNet(alpha=1, l1_ratio=.5)
```

Training the Model

- `fit()` teaches the model using the training data.

```
[17]: # Fit the model (i.e train the model)
model.fit(X_train, Y_train)
```

```
[17]: ElasticNet(alpha=1)
```

```
[18]: # evaluate the model
Y_pred = model.predict(X_test)
```

Evaluating the model

R² Score R² measures how well the model explains the data.

- 1.0 = perfect prediction
- 0.0 = same as predicting the average
- Negative = very bad model

For example: - If $R^2 = 0.87$, the model explains 87% of the differences in delivery time.

```
[19]: from sklearn.metrics import r2_score
r2_score(Y_test, Y_pred)
```

[19]: 0.9659303928407459

Mean Squared Error (MSE) MSE measures the average prediction error. - Lower is better. - Because the errors are squared, large mistakes are punished more strongly.

For example:

- If the model predicts 30 minutes but the real delivery time is 40 minutes, the error is: $(40 - 30)^2 = 100$

```
[20]: from sklearn.metrics import mean_squared_error  
mean_squared_error(Y_test, Y_pred)
```

[20]: 0.02468343038687958

```
[ ]:
```